Date Fruit Image Classification

Talal Aloushan
CSC462 - Machine Learning
Nov 2024

Contents

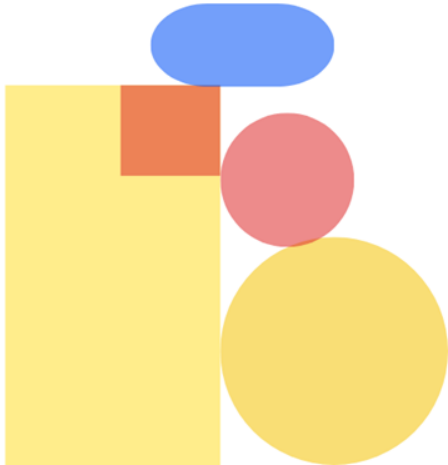
1. Project Overview
2. Problem Description and Importance
3. Platform and Dataset
4. CNN Approach Overview
5. Approach 1: Custom CNN Model
6. Approach 2: Transfer Learning with EfficientNetB0
7. Approach 3: YOLOv11n-cls Model
8. Conclusion

Project Overview

This presentation covers a machine learning project focused on classifying date fruit varieties. The aim is to solve the challenge of sorting dates quickly and accurately using computer vision, ensuring quality control, accurate pricing, and proper sorting in the agricultural industry.

Problem Description and Importance

In today's fast-paced agricultural industry, distinguishing different types of dates quickly and accurately is challenging. Manual sorting is time-consuming and error-prone. By using computer vision to categorize dates automatically, the process can be streamlined. This technology ensures consistent quality control, fair pricing, and proper sorting, benefiting farmers, processors, and consumers.



Dataset

1. Training Set
1156 samples

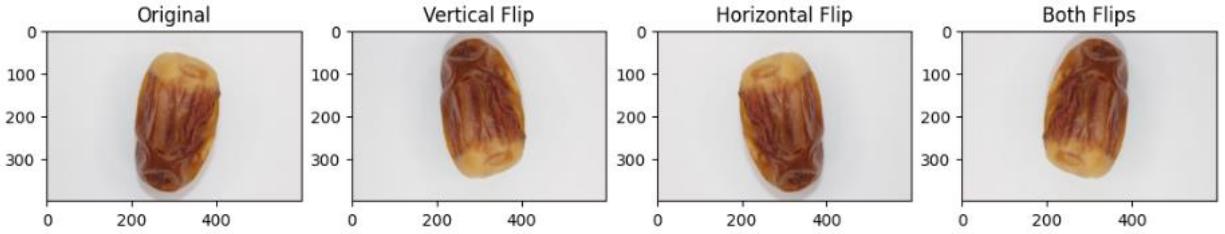
2. Validation Set
502 samples

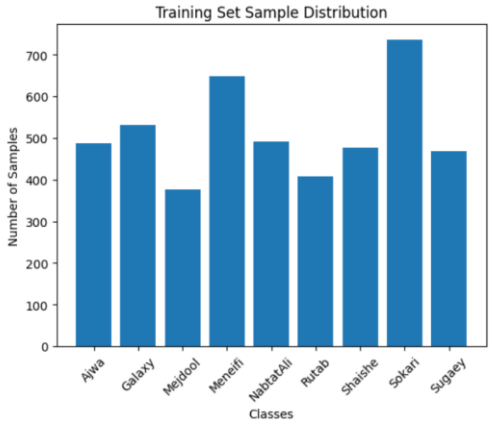
3. Image Size
600x400

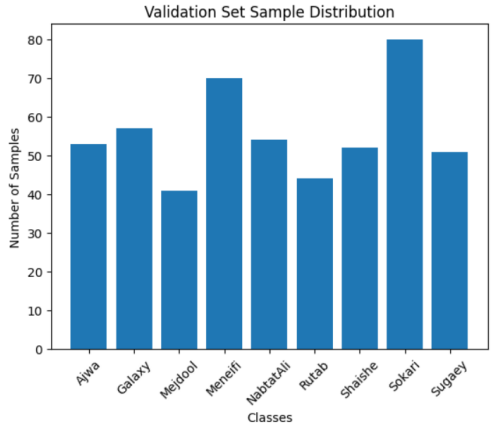


Dataset

After Augmentation
4624 samples for training.







CNN Approach Overview

Three approaches were explored for training a CNN model:

1. Custom CNN model using TensorFlow
2. Transfer learning with EfficientNetB0
3. Fine-tuning YOLOv11n-cls model

Approach 1: Custom CNN Model

Model Architecture and Training

Data Preprocessing

- Batched data into 32 batches
- Scaled pixel values to [0,1]
- Resized images to 200x200

Optimizer and Epochs:

- Adam with learning rate 0.00001
- Epochs: 300
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy

Architecture

Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 196, 196, 32)	2,432
max_pooling2d_9 (MaxPooling2D)	(None, 98, 98, 32)	0
conv2d_10 (Conv2D)	(None, 96, 96, 32)	9,248
max_pooling2d_10 (MaxPooling2D)	(None, 48, 48, 32)	0
conv2d_11 (Conv2D)	(None, 46, 46, 32)	9,248
max_pooling2d_11 (MaxPooling2D)	(None, 23, 23, 32)	0
flatten_3 (Flatten)	(None, 16928)	0
dense_6 (Dense)	(None, 256)	4,333,824
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 9)	2,313

Total params: 4,357,065

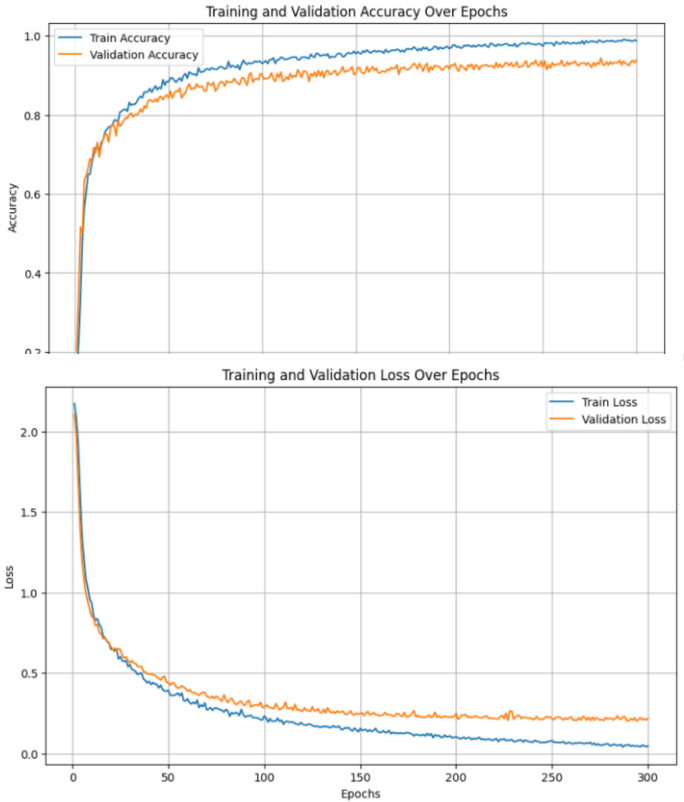
Approach 1: Custom CNN Model

Model Architecture and Training

Performance

Achieved 94% accuracy

```
145/145 ━━━━━━━━━━━━━━━━━━━━ 49s 338ms/step - accuracy: 0.9873 - loss: 0.0519 - val_accuracy: 0.9263 - val_loss: 0.2236
Epoch 274/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 342ms/step - accuracy: 0.9833 - loss: 0.0603 - val_accuracy: 0.9323 - val_loss: 0.2155
Epoch 275/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 341ms/step - accuracy: 0.9850 - loss: 0.0522 - val_accuracy: 0.9263 - val_loss: 0.2089
Epoch 276/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 339ms/step - accuracy: 0.9874 - loss: 0.0517 - val_accuracy: 0.9203 - val_loss: 0.2118
Epoch 277/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9854 - loss: 0.0552 - val_accuracy: 0.9323 - val_loss: 0.2060
Epoch 278/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9855 - loss: 0.0594 - val_accuracy: 0.9243 - val_loss: 0.2240
Epoch 279/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 339ms/step - accuracy: 0.9831 - loss: 0.0615 - val_accuracy: 0.9323 - val_loss: 0.2144
Epoch 280/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9893 - loss: 0.0507 - val_accuracy: 0.9283 - val_loss: 0.2190
Epoch 281/500
145/145 ━━━━━━━━━━━━━━━━━━━━ 50s 340ms/step - accuracy: 0.9866 - loss: 0.0498 - val_accuracy: 0.9442 - val_loss: 0.2162
```

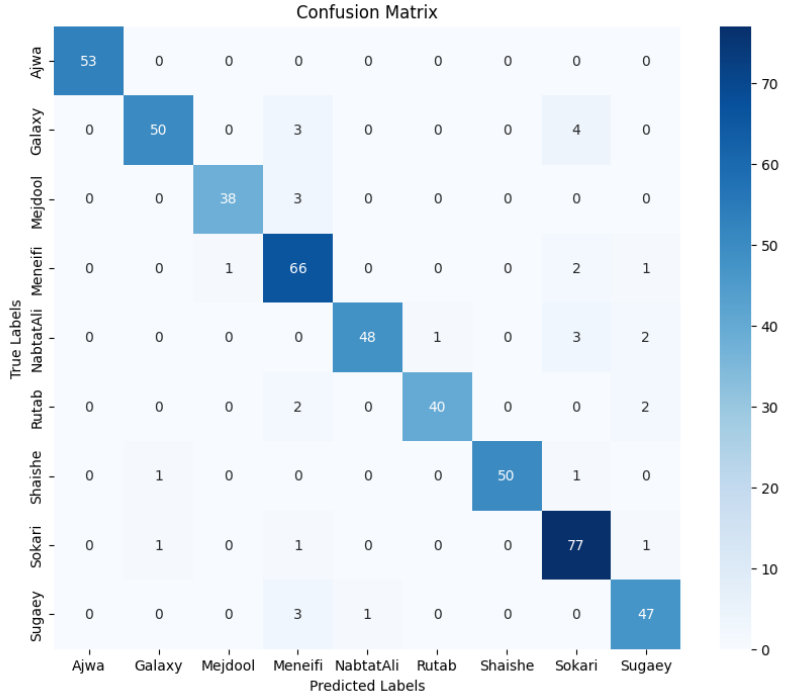


Approach 1: Custom CNN Model

Model Architecture and Training

Classification Report:

	precision	recall	f1-score	support
Ajwa	1.00	1.00	1.00	53
Galaxy	0.96	0.88	0.92	57
Mejdool	0.97	0.93	0.95	41
Meneifi	0.85	0.94	0.89	70
NabtatAli	0.98	0.89	0.93	54
Rutab	0.98	0.91	0.94	44
Shaishe	1.00	0.96	0.98	52
Sokari	0.89	0.96	0.92	80
Sugaey	0.89	0.92	0.90	51
accuracy			0.93	502
macro avg	0.95	0.93	0.94	502
weighted avg	0.94	0.93	0.93	502



Approach 2: EfficientNetB0

Frozen Model

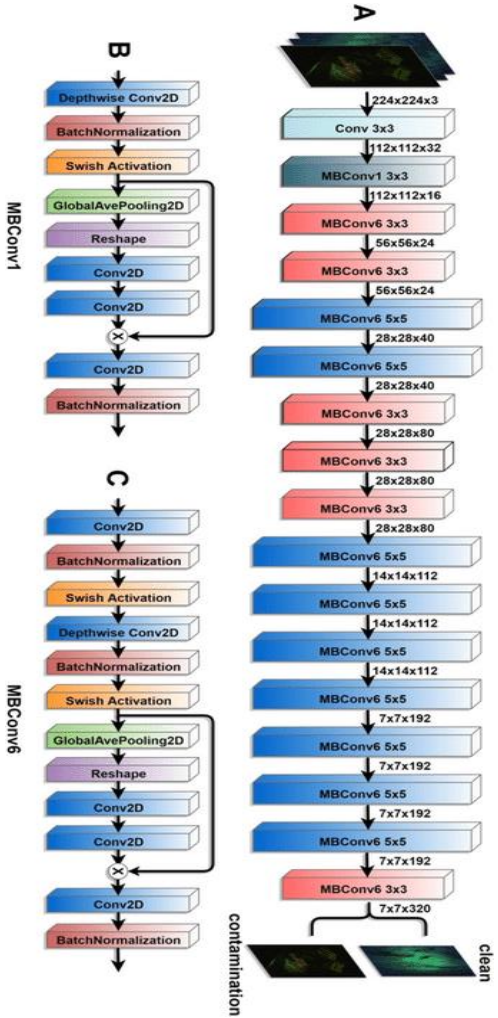
Data Preprocessing

- Batched data into 32 batches
- Scaled pixel values to [0,1]
- Resized images to 224x224

Architecture

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 9)	11,529

Total params: 4,061,100
Trainable params: 11,529



Approach 2: EfficientNetB0

Frozen Model

Optimizer and Epochs

- Adam with learning rate = 0.01
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy
- Epochs: 20

Performance

Achieved 96% accuracy

```
145/145 ━━━━━━━━━━━━━━━━━━━━ 128s 879ms/step - accuracy: 0.9645 - loss: 0.1095 - val_accuracy: 0.9522 - val_loss: 0.1374
Epoch 12/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 151s 1s/step - accuracy: 0.9651 - loss: 0.1004 - val_accuracy: 0.9482 - val_loss: 0.2386
Epoch 13/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 152s 1s/step - accuracy: 0.9653 - loss: 0.1077 - val_accuracy: 0.9582 - val_loss: 0.1535
Epoch 14/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 973ms/step - accuracy: 0.9681 - loss: 0.0954 - val_accuracy: 0.9382 - val_loss: 0.2326
Epoch 15/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 141s 970ms/step - accuracy: 0.9636 - loss: 0.0944 - val_accuracy: 0.9402 - val_loss: 0.2057
Epoch 16/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 141s 970ms/step - accuracy: 0.9621 - loss: 0.1251 - val_accuracy: 0.9422 - val_loss: 0.2029
Epoch 17/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 974ms/step - accuracy: 0.9606 - loss: 0.1246 - val_accuracy: 0.9482 - val_loss: 0.2323
Epoch 18/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 976ms/step - accuracy: 0.9703 - loss: 0.0813 - val_accuracy: 0.9622 - val_loss: 0.1516
Epoch 19/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 142s 976ms/step - accuracy: 0.9710 - loss: 0.0989 - val_accuracy: 0.9522 - val_loss: 0.1929
```

Approach 2: EfficientNetB0

Unfrozen Model

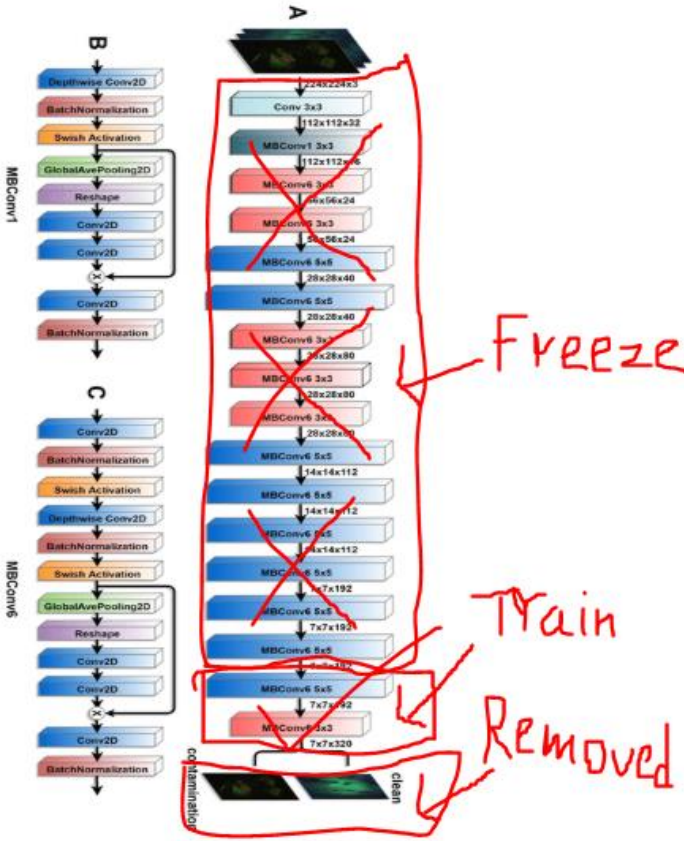
Data Preprocessing

- Batched data into 32 batches
- Scaled pixel values to [0,1]
- Resized images to 224x224

Architecture

Layer (type)	Output Shape	Param #
efficientnetb0 (Functional)	(None, 7, 7, 1280)	4,049,571
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 9)	11,529

Total params: 4,084,160
Trainable params: 1,354,297
Non-trainable params: 2,706,803



Approach 2: EfficientNetB0

Unfrozen Model

Optimizer and Epochs

- Adam with learning rate = 0.0001
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy
- Epochs: 20

Performance

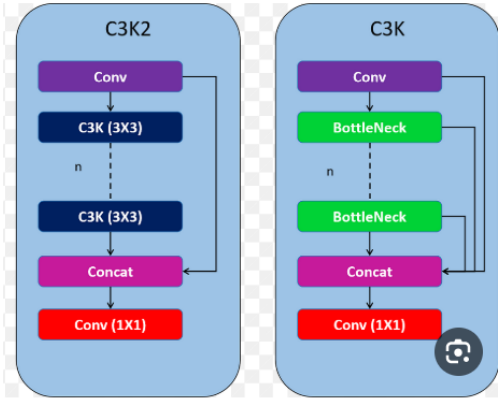
Achieved 97% accuracy

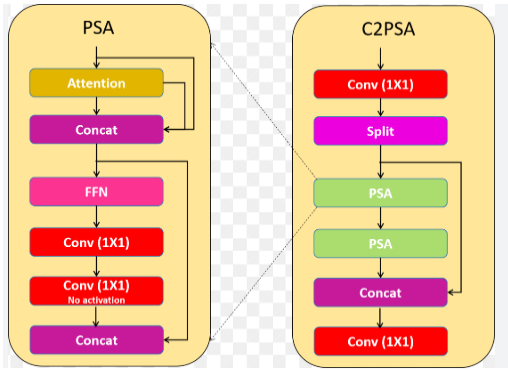
```
Epoch 13/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 162s 1s/step - accuracy: 0.9940 - loss: 0.0170 - val_accuracy: 0.9641 - val_loss: 0.1968
Epoch 14/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 164s 1s/step - accuracy: 0.9964 - loss: 0.0104 - val_accuracy: 0.9701 - val_loss: 0.1711
Epoch 15/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 162s 1s/step - accuracy: 0.9951 - loss: 0.0091 - val_accuracy: 0.9681 - val_loss: 0.1719
Epoch 16/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 157s 1s/step - accuracy: 0.9935 - loss: 0.0139 - val_accuracy: 0.9661 - val_loss: 0.1798
Epoch 17/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 159s 1s/step - accuracy: 0.9942 - loss: 0.0178 - val_accuracy: 0.9622 - val_loss: 0.1857
Epoch 18/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 157s 1s/step - accuracy: 0.9929 - loss: 0.0206 - val_accuracy: 0.9741 - val_loss: 0.1592
Epoch 19/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 160s 1s/step - accuracy: 0.9954 - loss: 0.0102 - val_accuracy: 0.9661 - val_loss: 0.1812
Epoch 20/50
145/145 ━━━━━━━━━━━━━━━━━━━━ 162s 1s/step - accuracy: 0.9974 - loss: 0.0077 - val_accuracy: 0.9641 - val_loss: 0.1728
```

Approach 3: YOLOv11n-cls Model

Data Preprocessing

- Batched data into 16 batches
- Scaled pixel values to [0,1]
- Resized images to 224x224





Architecture

from	n	params	module	arguments
-1	1	464	ultralytics.nn.modules.conv.Conv	[3, 16, 3, 2]
-1	1	4672	ultralytics.nn.modules.conv.Conv	[16, 32, 3, 2]
-1	1	6640	ultralytics.nn.modules.block.C3k2	[32, 64, 1, False, 0.25]
-1	1	36992	ultralytics.nn.modules.conv.Conv	[64, 64, 3, 2]
-1	1	26080	ultralytics.nn.modules.block.C3k2	[64, 128, 1, False, 0.25]
-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
-1	1	87040	ultralytics.nn.modules.block.C3k2	[128, 128, 1, True]
-1	1	295424	ultralytics.nn.modules.conv.Conv	[128, 256, 3, 2]
-1	1	346112	ultralytics.nn.modules.block.C3k2	[256, 256, 1, True]
-1	1	249728	ultralytics.nn.modules.block.C2PSA	[256, 256, 1]
-1	1	357141	ultralytics.nn.modules.head.Classify	[256, 21]

YOLO11n-cls summary: 151 layers, 1,558,005 parameters, 1,558,005 gradients, 3.3 GFLOPs

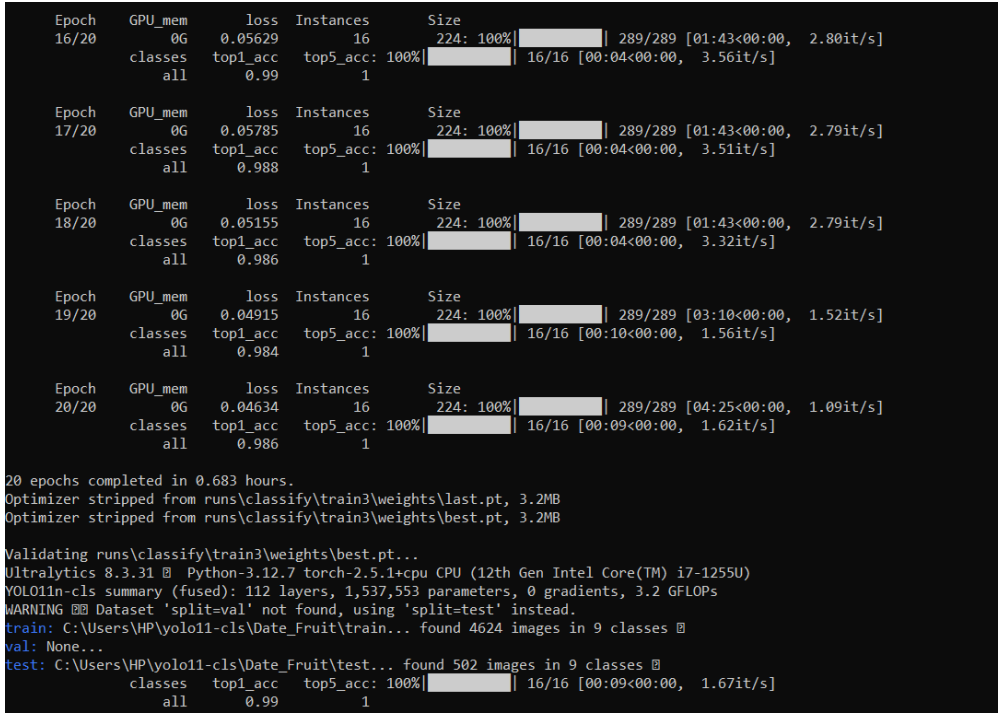
Approach 3: YOLOv11n-cls Model

Optimizer and Epochs

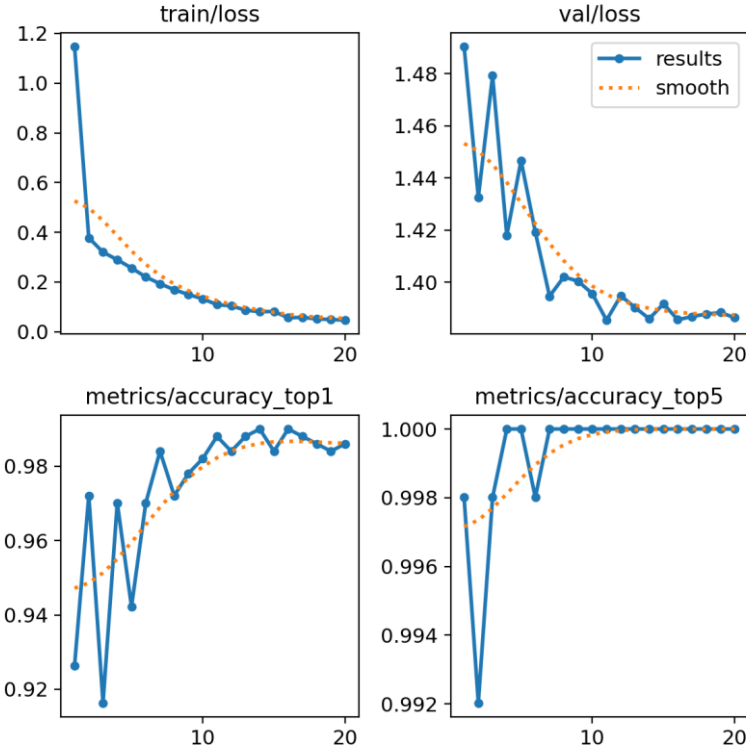
- AdamW with learning rate = 0.000714
- Loss Function: Categorical Crossentropy
- Metrics: Accuracy
- Epochs: 20

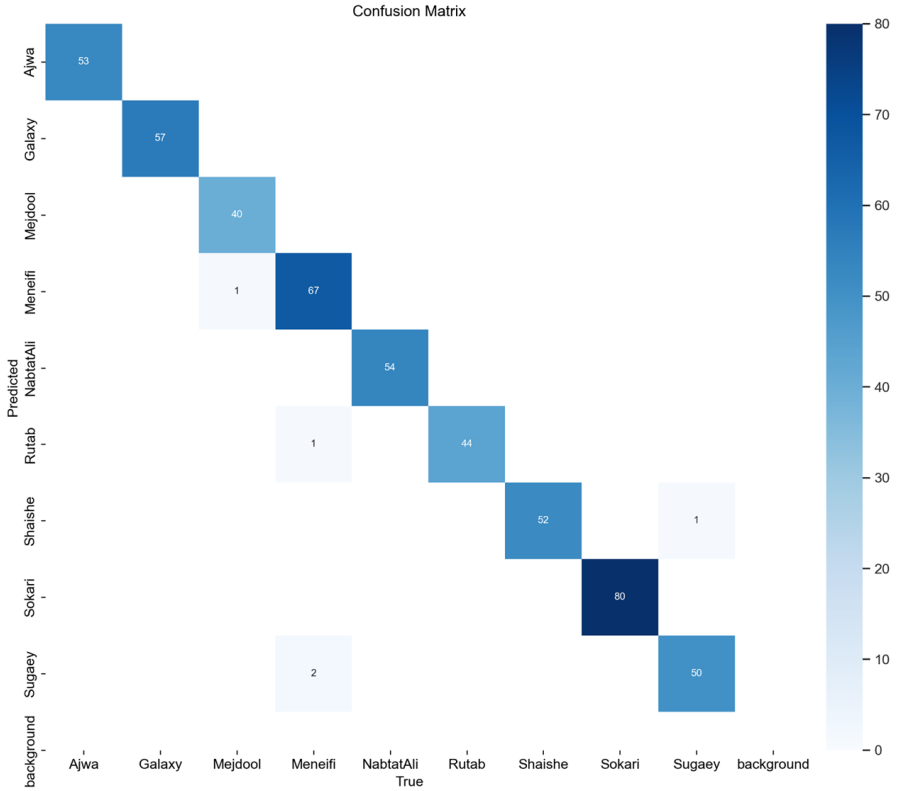
Performance

Achieved 99% accuracy



Approach 3: YOLOv11n-cls Model

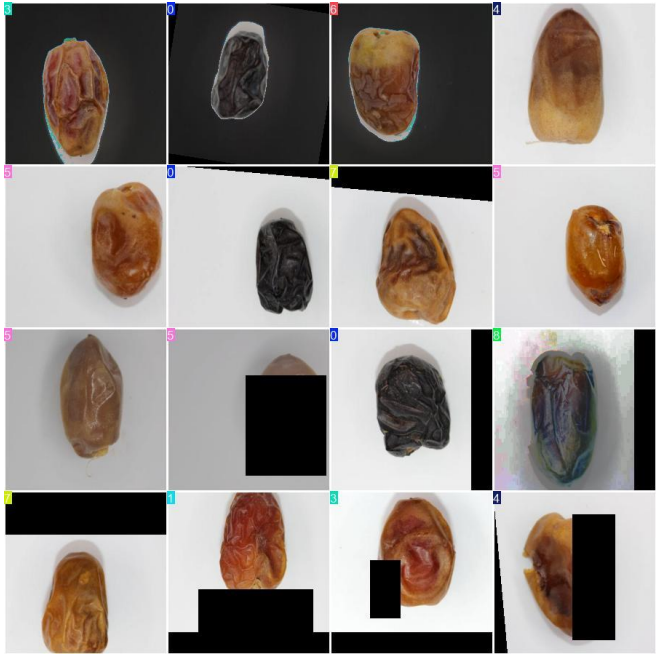




Approach 3: YOLOv11n-cls Model

YOLOv11-cls default random augmentations during training



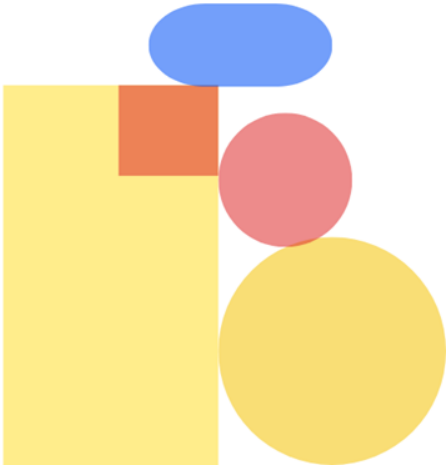


Comparison

Criteria	Custom CNN	EfficientNetB0	YOLO11n-cls
Number of Parameters	4.4M	4M	1.5M
Training Time	230 Minutes	45 Minutes	40 Minutes
Accuracy	94%	97%	99%
Implementation	Easy	Takes time	Very easy
Platform	TensorFlow	TensorFlow	PyTorch

Challenges

One of the challenges i had is the training time, when i was creating the custom CNN at first, i was having a very long time in each epoch so i reduced the model sized, also in EffecientnetB0 i tried to unfreeze 50% of the model but i got a long training time, so i decided to unfreeze 10% only, however i was using a cpu, if i used a gpu that would be a different case



Conclusion

In the project i successfully developed and trained models to classify date fruits with high accuracy. The custom CNN model achieved 94% accuracy, transfer learning with EfficientNetB0 reached up to 97%, and the YOLOv11n-cls model achieved 99% accuracy. This work demonstrates the potential of using computer vision in agriculture to improve sorting processes and quality control.

